

Методичка 4. Цикл `for`, `range`, `break` и `continue`

1. Зачем нужна эта тема

Цикл `for` нужен, когда программа должна повторить действие несколько раз.

Например:

- вывести числа от 1 до 10;
- посчитать сумму чисел;
- проверить несколько попыток пользователя;
- пройти по списку;
- вывести все буквы строки;
- выполнить действие ровно 5 раз.

Цикл `for` особенно удобен, когда количество повторений известно заранее.

Если цикл `while` чаще используется как “повторять, пока условие истинно”, то `for` обычно читается как “повторить для каждого значения”.

Примеры ситуаций:

- повторить действие 10 раз;
- перебрать числа от 1 до 100;
- пройти по всем элементам списка;
- пройти по всем символам строки.

Также в этой теме изучаются команды `break` и `continue`.

`break` позволяет досрочно остановить цикл. `continue` позволяет пропустить один шаг цикла и перейти к следующему.

2. Главная идея темы

Цикл `for` перебирает значения из последовательности.

Самый частый вариант для начала — перебор чисел через `range()`.

Цикл — повторение действий.

Итерация — один проход цикла.

range — команда, которая создаёт последовательность чисел.

break — команда для досрочной остановки цикла.

continue — команда для пропуска текущей итерации.

3. Базовый синтаксис

Простой цикл for

```
for i in range(5):  
    print(i)
```

Что происходит:

- `range(5)` создаёт числа от 0 до 4;
- переменная `i` по очереди принимает эти значения;
- на каждом шаге программа выводит `i`.

Результат:

```
0  
1  
2  
3  
4
```

На что обратить внимание:

- `range(5)` не включает число 5;
- счёт начинается с 0;
- тело цикла пишется с отступом.

range с началом и концом

```
for i in range(1, 6):  
    print(i)
```

Результат:

```
1  
2  
3  
4  
5
```

`range(1, 6)` означает: начать с 1, остановиться перед 6.

range с шагом

```
for i in range(2, 11, 2):  
    print(i)
```

Результат:

```
2  
4  
6  
8  
10
```

Третий аргумент — это шаг.

break

```
for i in range(1, 11):  
    if i == 5:  
        break  
  
    print(i)
```

Результат:

```
1  
2  
3  
4
```

Когда `i` становится 5, срабатывает `break`, и цикл полностью завершается.

continue

```
for i in range(1, 6):  
    if i == 3:  
        continue  
  
    print(i)
```

Результат:

```
1  
2  
4  
5
```

Когда `i` становится 3, срабатывает `continue`, и текущий шаг пропускается.

4. Разбор темы от простого к сложному

Шаг 1. Повторить действие несколько раз

```
for i in range(5):  
    print("Python")
```

Программа выводит слово `Python` 5 раз.

Здесь переменная `i` нужна только для подсчёта повторений.

Шаг 2. Вывести числа

```
for i in range(1, 11):  
    print(i)
```

Программа выводит числа от 1 до 10.

Важно: правую границу нужно ставить на 1 больше.

Шаг 3. Посчитать сумму чисел

```
total = 0  
  
for i in range(1, 6):  
    total = total + i  
  
print("Сумма:", total)
```

Что происходит:

- `total` начинается с 0;
- цикл перебирает числа от 1 до 5;
- каждое число прибавляется к `total`;
- после цикла выводится сумма.

Шаг 4. Вывести только чётные числа

```
for i in range(1, 11):  
    if i % 2 == 0:  
        print(i)
```

Программа проверяет каждое число. Если число делится на 2 без остатка, оно выводится.

Шаг 5. Использовать шаг в range

```
for i in range(2, 21, 2):  
    print(i)
```

Так можно вывести чётные числа от 2 до 20 без дополнительного условия.

Шаг 6. Досрочно остановить цикл

```
secret = 7  
  
for attempt in range(1, 4):  
    answer = int(input("Угадайте число: "))  
  
    if answer == secret:  
        print("Победа!")  
        break
```

Программа даёт пользователю максимум 3 попытки. Если число угадано раньше, цикл останавливается.

Шаг 7. Пропустить ненужные значения

```
for i in range(1, 11):  
    if i % 2 != 0:  
        continue  
  
    print(i)
```

Если число нечётное, программа пропускает текущий шаг. Печатаются только чётные числа.

Шаг 8. Перебор строки

```
word = "python"  
  
for letter in word:  
    print(letter)
```

Цикл `for` умеет перебирать не только числа, но и символы строки.

Результат:

```
p  
y  
t  
h  
o  
n
```

5. Частые ошибки учеников

Ошибка 1. Думают, что `range(10)` даёт числа от 1 до 10

Неправильно:

```
for i in range(10):  
    print(i)
```

Почему это ошибка:

Код не ошибочный технически, но он выводит числа от 0 до 9, а не от 1 до 10.

Правильно:

```
for i in range(1, 11):  
    print(i)
```

Ошибка 2. Забывают отступ внутри цикла

Неправильно:

```
for i in range(5):  
print(i)
```

Почему это ошибка:

Python понимает тело цикла по отступу.

Правильно:

```
for i in range(5):  
    print(i)
```

Ошибка 3. Ставят правую границу без запаса

Неправильно:

```
for i in range(1, 10):  
    print(i)
```

Почему это ошибка:

Если нужно вывести числа от 1 до 10, такой код выведет только от 1 до 9.

Правильно:

```
for i in range(1, 11):  
    print(i)
```

Ошибка 4. Меняют переменную цикла вручную

Неправильно:

```
for i in range(1, 6):  
    print(i)  
    i = i + 1
```

Почему это ошибка:

В цикле `for` переменная `i` меняется автоматически. Ручное изменение обычно не влияет на следующий шаг так, как ожидается.

Правильно:

```
for i in range(1, 6):  
    print(i)
```

Ошибка 5. Путают `break` и `continue`

Неправильно:

```
for i in range(1, 6):  
    if i == 3:  
        break  
  
    print(i)
```

Почему это ошибка:

Если нужно только пропустить число 3, `break` не подходит. Он полностью остановит цикл.

Правильно:

```
for i in range(1, 6):  
    if i == 3:  
        continue  
  
    print(i)
```

Ошибка 6. Создают накопитель внутри цикла

Неправильно:

```
for i in range(1, 6):  
    total = 0  
    total = total + i  
  
print(total)
```

Почему это ошибка:

`total` каждый раз снова становится 0, поэтому сумма не накапливается.

Правильно:

```
total = 0

for i in range(1, 6):
    total = total + i

print(total)
```

Ошибка 7. Используют for там, где лучше while

Неправильно:

```
for i in range(100):
    password = input("Введите пароль: ")

    if password == "1234":
        break
```

Почему это не лучший вариант:

Код работает, но количество попыток выбрано случайно. Если программа должна спрашивать пароль до правильного ввода без ограничения, лучше использовать `while`.

Правильно:

```
password = ""

while password != "1234":
    password = input("Введите пароль: ")

print("Вход выполнен")
```

6. Мини-примеры для закрепления

Пример 1. Повторение текста

```
for i in range(3):
    print("Привет")
```

Программа выводит текст 3 раза. Закрепляется простое повторение через `for`.

Пример 2. Числа от 1 до 10

```
for i in range(1, 11):  
    print(i)
```

Закрепляется работа с границами `range` .

Пример 3. Чётные числа

```
for i in range(2, 21, 2):  
    print(i)
```

Закрепляется шаг в `range` .

Пример 4. Сумма чисел

```
total = 0  
  
for i in range(1, 101):  
    total = total + i  
  
print(total)
```

Программа считает сумму от 1 до 100. Закрепляется накопитель.

Пример 5. Поиск числа

```
for i in range(1, 11):  
    if i == 7:  
        print("Нашли число")  
        break
```

Закрепляется `break` .

Пример 6. Пропуск отрицательных чисел

```
for i in range(-3, 4):  
    if i < 0:  
        continue  
  
    print(i)
```

Программа выводит только 0 и положительные числа. Закрепляется `continue` .

Пример 7. Перебор букв

```
word = "game"

for letter in word:
    print(letter)
```

Закрепляется перебор строки через `for` .

7. Практические задания

Лёгкий уровень

1. Вывести числа от 1 до 10.
2. Вывести числа от 10 до 1.
3. Вывести все чётные числа от 2 до 20.
4. Вывести слово `Python` 7 раз.
5. Посчитать сумму чисел от 1 до 50.

Средний уровень

1. Пользователь вводит число `n` . Вывести числа от 1 до `n` .
2. Пользователь вводит число `n` . Посчитать сумму чисел от 1 до `n` .
3. Вывести все числа от 1 до 100, которые делятся на 5.
4. Дать пользователю 3 попытки ввести правильный пароль.
5. Пользователь вводит слово. Вывести каждую букву на новой строке.

Повышенный уровень

1. Пользователь вводит число `n` . Посчитать сумму только чётных чисел от 1 до `n` .
2. Пользователь вводит 5 чисел. Найти сумму положительных чисел.
3. Сделать игру “угадай число” с ограничением в 5 попыток.

8. Контрольные вопросы

1. Для чего нужен цикл `for` ?
2. Чем `for` отличается от `while` ?
3. Что делает `range(5)` ?
4. Почему `range(1, 10)` не включает 10?
5. Зачем нужен третий аргумент в `range` ?

6. Что такое итерация?
7. Что делает `break` ?
8. Что делает `continue` ?
9. В каком случае лучше использовать `break` ?
10. В каком случае лучше использовать `continue` ?
11. Почему накопитель нужно создавать до цикла?
12. Можно ли циклом `for` перебирать строку?

9. Краткий конспект в конце

Главная идея темы: цикл `for` повторяет действия для каждого значения из последовательности.

Основные конструкции:

```
for i in range(5):  
    print(i)
```

```
for i in range(1, 11):  
    print(i)
```

```
for i in range(2, 21, 2):  
    print(i)
```

```
for i in range(10):  
    if i == 5:  
        break
```

```
for i in range(10):  
    if i == 5:  
        continue
```

Важно запомнить:

- `range(5)` даёт числа от 0 до 4;
- правая граница не включается;
- `break` полностью останавливает цикл;
- `continue` пропускает только текущий шаг;
- `for` удобно использовать, когда количество повторений известно заранее;
- накопители создаются до цикла.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- писать цикл `for` ;
- использовать `range` ;
- задавать начало, конец и шаг;
- считать сумму через цикл;
- выводить последовательности чисел;
- использовать `break` ;
- использовать `continue` ;
- понимать разницу между `for` и `while` ;
- решать задачи с известным количеством повторений.